GUIDE 10

Deploy to a DigitalOcean VPS with Docker and GitHub Actions

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Create and secure an Ubuntu DigitalOcean Droplet.
- Containerize React, Spring Boot and MySQL for a single-VPS monolithic deployment.
- Configure Nginx, persistent storage and environment secrets.
- Build CI/CD workflows that test, publish images and deploy through SSH.
- Operate the deployment with backups, health checks, rollback and cost controls.

Prerequisites

- Frontend and backend repositories pass local tests.
- A GitHub account and DigitalOcean account.
- A domain is required for trusted public HTTPS.

Suggested student time: 4-5 hours

1. Target deployment architecture

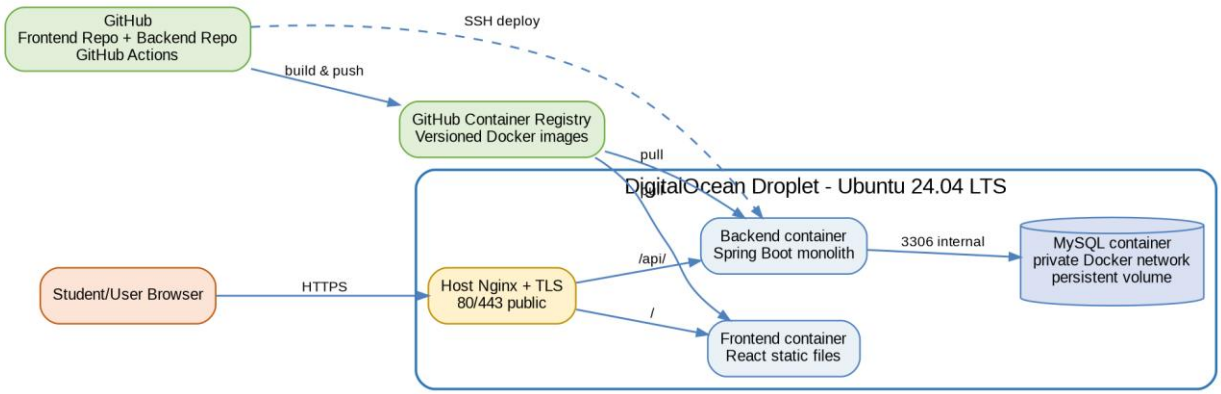


Figure 1. Single-Droplet deployment with separate containers but one monolithic backend.

Architecture meaning: Containers do not turn the backend into microservices. The Spring Boot application remains one deployable monolith connected to one MySQL database.

2. Create the Droplet

1. Verify any student credit and billing terms before creating resources. Configure a billing alert and delete unused resources after assessment.
2. Create an Ubuntu 24.04 LTS Droplet in a nearby region. Begin with a small Basic plan suitable for a class demo; monitor memory before increasing size.
3. Use SSH-key authentication. Do not enable password-only root access.
4. Assign the Droplet to a project and use a clear name such as unimart-team-03.
5. Record the public IP and connect using ssh root@SERVER_IP.

3. Secure the server first

```
[Ubuntu terminal]
adduser deploy
usermod -aG sudo deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy

ufw allow OpenSSH
ufw enable
ufw status verbose

apt update && apt upgrade -y
```

Open a second terminal and verify ssh deploy@SERVER_IP works before closing the root session. Disable password authentication only after key access is confirmed and console recovery is understood.

4. Install Docker Engine and Compose plugin

Follow the current official Docker Engine for Ubuntu repository instructions rather than installing obsolete docker-compose packages. Verify with:

```
[Ubuntu terminal]
sudo systemctl status docker
sudo docker run hello-world
sudo docker compose version
sudo usermod -aG docker deploy
```

Docker privilege: Membership in the docker group grants root-equivalent control over the host. Limit membership and protect the deploy account's SSH key.

5. Backend Dockerfile

```
[Dockerfile]
# unimart-backend/Dockerfile
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw -B -DskipTests dependency:go-offline
COPY src src
RUN ./mvnw -B clean package -DskipTests

FROM eclipse-temurin:21-jre
RUN useradd --system --uid 10001 spring
WORKDIR /app
COPY --from=build /workspace/target/*.jar app.jar
USER spring
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app/app.jar"]
```

6. Frontend Dockerfile

```
[Dockerfile]
# unimart-frontend/Dockerfile
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE_URL=/api/v1
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
RUN npm run build

FROM nginx:alpine
COPY nginx-spa.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

```
[Nginx]
# nginx-spa.conf inside frontend image
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }

  location ~* \.(js|css|png|jpg|jpeg|gif|svg|woff2)$ {
    expires 7d;
    add_header Cache-Control "public, immutable";
  }
}
```

7. Docker Compose on the VPS

```
[YAML]
# /opt/unimart/docker-compose.yml
services:
  db:
    image: mysql:8.4
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: unimart
      MYSQL_USER: unimart_app
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    networks: [unimart]
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -p$${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10

  backend:
    image: ghcr.io/ORG/unimart-backend:latest
    restart: unless-stopped
    environment:
      SPRING_PROFILES_ACTIVE: prod
      DB_URL: jdbc:mysql://db:3306/unimart
      DB_USERNAME: unimart_app
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
      APP_ALLOWED_ORIGINS: ${APP_ALLOWED_ORIGINS}
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "127.0.0.1:8080:8080"
    networks: [unimart]

  frontend:
    image: ghcr.io/ORG/unimart-frontend:latest
    restart: unless-stopped
    ports:
      - "127.0.0.1:3000:80"
    networks: [unimart]

networks:
  unimart:

volumes:
  mysql_data:
```

```
[Environment]
# /opt/unimart/.env - chmod 600; never commit
DB_PASSWORD=replace-with-strong-random-value
MYSQL_ROOT_PASSWORD=replace-with-different-random-value
JWT_SECRET=replace-with-base64-secret
APP_ALLOWED_ORIGINS=https://unimart.example.edu
```

8. Host Nginx reverse proxy

```
[/etc/nginx/sites-available/unimart]
server {
  listen 80;
  server_name unimart.example.edu;

  location /api/ {
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

```
[Ubuntu terminal]
sudo apt install nginx -y
sudo ln -s /etc/nginx/sites-available/unimart /etc/nginx/sites-enabled/unimart
sudo nginx -t
sudo systemctl reload nginx
sudo ufw allow 'Nginx Full'
```

Point the domain's A record to the Droplet, then use the current Certbot Nginx instructions to obtain and automatically renew a trusted TLS certificate. Without a domain/TLS certificate, use only non-sensitive demonstration data.

9. Prepare GitHub Container Registry access

- Workflows use GITHUB_TOKEN to push their own repository package to GHCR.
- On the server, authenticate once with a narrowly scoped read:packages token if images are private.
- Use package visibility and organization permissions deliberately.
- Prefer immutable version/SHA tags for rollback; latest is only a convenience pointer.

10. Backend GitHub Actions workflow

```yaml
[GitHub Actions YAML]
name: backend-ci-cd
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  packages: write

jobs:
  test-build-publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: ./mvnw -B verify
      - if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - if: github.event_name == 'push'
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${{ github.repository_owner }}/unimart-backend:latest
            ghcr.io/${{ github.repository_owner }}/unimart-backend:${{ github.sha }}

  deploy:
    if: github.event_name == 'push'
    needs: test-build-publish
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Configure SSH
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.DO_SSH_PRIVATE_KEY }}" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan -H "${{ secrets.DO_HOST }}" >> ~/.ssh/known_hosts
      - name: Deploy backend
        run: |
          ssh "${{ secrets.DO_USER }}@${{ secrets.DO_HOST }}" \
            'cd /opt/unimart && docker compose pull backend && \
             docker compose up -d backend && docker image prune -f'
```

11. Frontend GitHub Actions workflow

```
[GitHub Actions YAML]
name: frontend-ci-cd
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  packages: write

jobs:
  test-build-publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v7
        with:
          node-version: '22'
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run build
      - if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - if: github.event_name == 'push'
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          build-args: VITE_API_BASE_URL=/api/v1
          tags: |
            ghcr.io/${{ github.repository_owner }}/unimart-frontend:latest
            ghcr.io/${{ github.repository_owner }}/unimart-frontend:${{ github.sha }}

  deploy:
    if: github.event_name == 'push'
    needs: test-build-publish
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.DO_SSH_PRIVATE_KEY }}" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          ssh-keyscan -H "${{ secrets.DO_HOST }}" >> ~/.ssh/known_hosts
      - run: |
          ssh "${{ secrets.DO_USER }}@${{ secrets.DO_HOST }}" \
            'cd /opt/unimart && docker compose pull frontend && \
             docker compose up -d frontend && docker image prune -f'
```

Pipeline hardening: For production, pin third-party actions to reviewed full commit SHAs and protect the production environment with required approval. The version tags above are easier for teaching but are mutable references.

12. GitHub secrets and environment

Secret	Value
DO_HOST	Droplet IP or hostname.
DO_USER	deploy.
DO_SSH_PRIVATE_KEY	Dedicated CI deploy key; not a personal primary key.
Optional deployment variables	Non-secret domain/image names can be environment variables.

- Create a GitHub environment named production and restrict it to main.
- Use manual approval for student demonstrations where supported.
- Never pass DB_PASSWORD or JWT_SECRET through workflow logs; they remain only in /opt/unimart/.env on the server or a secret manager.
- Rotate the deploy key when a team member leaves or the exercise ends.

13. Deploy, verify and roll back

```
[VPS terminal]
cd /opt/unimart
docker compose pull
docker compose up -d
docker compose ps
docker compose logs --tail=100 backend
curl -f http://127.0.0.1:8080/actuator/health
curl -I http://127.0.0.1:3000
```

```
[Rollback]
# Roll back by pinning the previous SHA tag in docker-compose.yml
# then:
docker compose pull backend frontend
docker compose up -d backend frontend
```

14. Operations, backup and cost control

- Back up MySQL daily. Test restore; a backup that has never been restored is unproven.
- Enable DigitalOcean monitoring/alerts and inspect memory, disk and CPU. Small Droplets can fail from memory pressure during image builds; build in GitHub Actions, not on the VPS.
- Keep MySQL unexposed; only backend accesses it on the Docker network.
- Apply OS and container updates deliberately, with a rollback point.
- Limit logs and Docker images so disk does not fill.
- Delete Droplets, volumes, snapshots and reserved IPs after the course unless explicitly retained.
- Do not use real card data, personal data or production credentials in the student deployment.

Completion checklist

☐ Droplet uses Ubuntu 24.04 LTS, SSH keys, non-root deploy user and firewall.

☐ Docker/Compose are installed from official instructions.

☐ Frontend, backend and database run in Compose; database has a persistent volume.

☐ Only Nginx ports and SSH are public; backend/frontend bindings are localhost-only.

☐ Domain and HTTPS are configured for any public authenticated deployment.

☐ Both repositories test, build and publish images before deployment.

☐ Production GitHub environment protects SSH secrets.

☐ Health checks, backup, rollback and resource deletion procedures are documented.